ForgeRoom — Complete Technical Architecture Document

Hackathon Execution Blueprint v2.0

Table of Contents

1. Winning Strategy & Reframe
2. System Architecture Overview
3. Tech Stack (Locked)
4. Data Contracts — The Hour 1 Lock
5. Backend 1 — Orchestrator Service (FastAPI + LangGraph)
6. Backend 2 — Execution Bridge (FastAPI + Subprocess + Git)
7. Backend 3 — Multiplayer State (FastAPI WebSockets)
8. The Blame Graph — Full Specification
9. Architectural Drift Detection — Full Specification
10. Risk Autopsy Report — Full Specification
11. Frontend Architecture (Next.js 15)
12. LangGraph State Machine — Deep Dive
13. Prompt Engineering — All Prompts
14. Dummy Repository Setup
15. 24-Hour Build Plan — Hour by Hour
16. Team Responsibilities — Exact Deliverables
17. Demo Script — Word for Word
18. Failure Modes & Fallbacks

---

1. Winning Strategy & Reframe

The Pitch (Memorize This)

> "Every project fails the same way — not because the code was wrong, but because the decisions that shaped the code were never challenged. ForgeRoom is the first tool that gives your AI the power to veto a bad architectural decision before a single line of code is written."

Why This Wins

Most hackathon projects are features. ForgeRoom is a **paradigm**. The three additions below are what separate a finalist from a winner:

Feature	Why It Wins
Blame Graph	Visual, real-time, makes judges gasp. No other team will have a live dependency graph of decisions.
Architectural Drift Detection	Technically hard, genuinely useful. Scans actual code and cross-references decisions. Nobody else does this.
Risk Autopsy	Meta-intelligence. The tool judges its own output. That's a level of AI sophistication most teams can't claim.

What We Are NOT Building

- QR code room sharing (cut it, waste of time)
- Time Saved Ticker (fake metric, hurts credibility)
- Skill URL pasting (too niche, too fragile for demo)

---

2. System Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                                   BROWSER CLIENTS                                            │
│                                                                                              │
│  ┌──────────────────────────────────────────────────────────────────────────────────────────┐
│  │  Next.js 15 Frontend                                                                     │
│  │  ┌──────────────┐  ┌──────────────────┐  ┌──────────────────────────────┐                │
│  │  │ Chat UI      │  │ Blame Graph      │  │ Living Spec Panel            │                │
│  │  │              │  │ (React Flow)     │  │                              │                │
│  │  └──────────────┘  └──────────────────┘  └──────────────────────────────┘                │
│  │                                                                                          │
│  │                                                                                          │
│  └──────────────────────────────────────────────────────────────────────────────────────────┘
│                                                                                              │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │ WebSocket (ws://) + REST (HTTP)
                                  ▼
┌──────────────────────────────────────────────────────────────────────────────────────────────┐
│                   Backend 3: WebSocket Gateway (Port 8002)                                   │
│                   FastAPI — Connection Manager — Broadcaster                                 │
│                   Debounce Buffer (3-5s asyncio)                                             │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  ┌──────────────────────────────────────────────────────────┐
```

```
        │ HTTP            │ HTTP
┌──────────────▼──────────────┐
┌──────────────▼──────────────────────────────────────────┐
│ Backend 1:       │  │ Backend 2:                          │
│ Orchestrator     │  │ Execution Bridge (Port 8001)        │
│ (Port 8000)      │  │                                     │
│                  │  │ • subprocess.run(gemini CLI)        │
│ • LangGraph      │  │ • Diff generation                   │
│ • Gemini Free    │  │ • git apply / git push              │
│ • Drift Detect   │  │ • Codebase snapshot reader          │
│ • Risk Autopsy   │  │                                     │
└──────────────┬──────────────┘
└──────────────────────────────────────────────────────────┘
        │ Gemini API (Free Tier)
┌──────────────▼──────────────────────────────────────────────────────────────────────────────┐
│                    Google Gemini API                                                        │
│     Free Tier (Supervisor/Drift)    Pro Tier (Code Gen CLI)                                 │
└──────────────────────────────────────────────────────────────────────────────────────────────┘
```

Port Map

Service	Port	Responsibility
Backend 1 (Orchestrator)	8000	LangGraph, Gemini calls, Drift Detection, Risk Autopsy
Backend 2 (Execution)	8001	Gemini CLI, git, diff apply
Backend 3 (WebSocket)	8002	Real-time state, broadcasting, debounce
Frontend (Next.js)	3000	All UI

---

3. Tech Stack (Locked)

Frontend

Next.js 15 (App Router)
React 19
Tailwind CSS
shadcn/ui (components)
React Flow (reactflow) — for the Blame Graph
react-markdown — for Living Spec rendering
zustand — lightweight global state

Backend (all Python 3.11+)

FastAPI
uvicorn
LangGraph
langchain-google-genai
websockets (built into FastAPI)
asyncio
subprocess (stdlib)
gitpython — cleaner git operations than raw subprocess
python-dotenv

Install Commands

```
# Frontend
npx create-next-app@latest forgeroom --typescript --tailwind --app
npx shadcn@latest init
npm install reactflow react-markdown zustand

# Backend
pip install fastapi uvicorn langgraph langchain-google-genai \
            gitpython python-dotenv aiofiles
```

---

4. Data Contracts — The Hour 1 Lock

ALL FOUR TEAM MEMBERS MUST AGREE ON THESE BEFORE WRITING A SINGLE LINE OF CODE.

4.1 WebSocket Message Schema

Every WebSocket message is a JSON object with this envelope:

```
interface WSMessage {
  type: "chat" | "conflict" | "spec_update" | "blame_graph" |
        "drift_alert" | "vote_cast" | "vote_result" | "agent_response" |
        "status" | "error";
  room_id: string;
  sender_id: string;         // "user:{name}" or "system:supervisor" or "agent:{name}"
  timestamp: string;         // ISO 8601
  payload: ChatPayload | ConflictPayload | SpecPayload |
           BlameGraphPayload | DriftAlertPayload | AgentPayload;
}
```

ChatPayload

```
interface ChatPayload {
  message: string;
  display_name: string;
}
```

ConflictPayload

```
interface ConflictPayload {
  conflict_id: string;       // uuid4
  summary: string;           // Human-readable description of the conflict
  option_a: string;          // First option
  option_b: string;          // Second option
  context: string;           // Which decision triggered this
  votes: { [user_id: string]: "a" | "b" };  // updated in real time
  resolved: boolean;
  winner: "a" | "b" | null;
}
```

SpecPayload

```
interface SpecPayload {
  current_goal: string;
  approved_decisions: Decision[];
  pending_tasks: string[];
  open_conflicts: number;
}

interface Decision {
  id: string;                // uuid4
  timestamp: string;
  description: string;
  category: "auth" | "database" | "api" | "frontend" | "infra" | "security" | "general";
  depends_on: string[];      // list of decision IDs this depends on
  contradicts: string[];     // list of decision IDs this contradicts (populated by Drift Detector)
  risk_score: number;        // 0-10, populated by Risk Autopsy
}
```

BlameGraphPayload

```
interface BlameGraphPayload {
  nodes: BlameNode[];
  edges: BlameEdge[];
}

interface BlameNode {
  id: string;                // decision ID
  label: string;             // short description
  category: string;
```

```
  risk_score: number;        // drives node color: 0-3 green, 4-6 yellow, 7-10 red
  is_new: boolean;           // triggers animation on frontend
}

interface BlameEdge {
  id: string;
  source: string;            // decision ID
  target: string;            // decision ID
  type: "depends_on" | "contradicts";
  // type "contradicts" renders as a red dashed edge
}
```

DriftAlertPayload

```
interface DriftAlertPayload {
  drift_id: string;
  proposed_decision: string;
  conflicting_file: string;     // e.g. "src/auth/middleware.py"
  conflicting_line: number;     // e.g. 47
  conflicting_code_snippet: string;
  explanation: string;          // e.g. "This decision contradicts existing session-cookie auth on
line 47"
  severity: "low" | "medium" | "high";
}
```

AgentPayload

```
interface AgentPayload {
  agent_name: string;           // e.g. "AppSec"
  agent_emoji: string;          // e.g. "🛡️"
  response: string;
  referenced_files: string[];   // files the agent read
}
```

4.2 REST Endpoints

```
POST /api/rooms                    → Create room, returns room_id
GET  /api/rooms/{room_id}          → Get room state
POST /api/rooms/{room_id}/vote     → Cast vote on conflict
POST /api/execute-spec             → Trigger Gemini CLI code generation
POST /api/apply-diff               → Apply diff + git push
GET  /api/rooms/{room_id}/export   → Download session markdown
POST /api/rooms/{room_id}/agent    → Invoke a specific agent skill
POST /api/rooms/{room_id}/drift-check → Trigger drift detection on a decision
```

4.3 LangGraph State Object

```
from typing import TypedDict, List, Optional
from dataclasses import dataclass, field
import uuid

@dataclass
class Decision:
    id: str = field(default_factory=lambda: str(uuid.uuid4()))
    description: str = ""
    category: str = "general"
    depends_on: List[str] = field(default_factory=list)
    contradicts: List[str] = field(default_factory=list)
    risk_score: float = 0.0
    timestamp: str = ""

class ForgeRoomState(TypedDict):
    room_id: str
    chat_history: List[dict]          # raw message dicts [{sender, message, timestamp}]
    approved_decisions: List[Decision]
    pending_conflicts: List[dict]      # ConflictPayload dicts
    current_goal: str
    pending_tasks: List[str]
    blame_graph_nodes: List[dict]
    blame_graph_edges: List[dict]
    last_drift_alerts: List[dict]
    session_start: str
    focus_mode: bool                   # if True, supervisor reads but does not respond
```

---

5. Backend 1 — Orchestrator Service (FastAPI + LangGraph)

File Structure

```
orchestrator/
├── main.py              # FastAPI app, route definitions
├── state.py             # ForgeRoomState TypedDict
├── graph.py             # LangGraph graph definition
├── nodes/
│   ├── supervisor.py      # Conflict detection + spec update
│   ├── drift_detector.py  # Architectural Drift Detection logic
│   ├── risk_autopsy.py    # Post-session risk report generator
│   └── agents/
│       └── appsec.py      # AppSec agent node
├── prompts.py           # All prompt strings (see Section 13)
└── utils.py             # Codebase snapshot reader
```

main.py (Core Routes)

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from graph import ForgeRoomGraph
from state import ForgeRoomState
import asyncio, uuid
from datetime import datetime

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

# In-memory room store (fine for hackathon)
rooms: dict[str, ForgeRoomState] = {}
graphs: dict[str, ForgeRoomGraph] = {}

class ChatBatch(BaseModel):
    room_id: str
    messages: list[dict]

class DriftCheckRequest(BaseModel):
    room_id: str
    proposed_decision: str
    category: str

@app.post("/api/rooms")
async def create_room(body: dict):
    room_id = str(uuid.uuid4())[:8]
    rooms[room_id] = ForgeRoomState(
        room_id=room_id,
        chat_history=[],
        approved_decisions=[],
        pending_conflicts=[],
        current_goal="No goal set yet",
        pending_tasks=[],
        blame_graph_nodes=[],
        blame_graph_edges=[],
        last_drift_alerts=[],
        session_start=datetime.utcnow().isoformat(),
        focus_mode=False,
    )
    graphs[room_id] = ForgeRoomGraph(room_id)
    return {"room_id": room_id}

@app.post("/api/process-batch")
async def process_chat_batch(body: ChatBatch):
```

```
    """Called by WebSocket backend after debounce fires."""
    if body.room_id not in rooms:
        raise HTTPException(404, "Room not found")

    state = rooms[body.room_id]
    state["chat_history"].extend(body.messages)

    graph = graphs[body.room_id]
    result = await graph.run(state)
    rooms[body.room_id] = result
    return result

@app.post("/api/rooms/{room_id}/drift-check")
async def drift_check(room_id: str, body: DriftCheckRequest):
    from nodes.drift_detector import run_drift_detection
    result = await run_drift_detection(body.proposed_decision, body.category)
    return result

@app.get("/api/rooms/{room_id}/export")
async def export_session(room_id: str):
    from utils import generate_markdown_export
    state = rooms[room_id]
    md = generate_markdown_export(state)
    return {"markdown": md}
```

graph.py — LangGraph Definition

```
from langgraph.graph import StateGraph, END
from state import ForgeRoomState
from nodes.supervisor import supervisor_node
from nodes.drift_detector import drift_detector_node
from nodes.risk_autopsy import risk_autopsy_node

class ForgeRoomGraph:
    def __init__(self, room_id: str):
        self.room_id = room_id
        self.graph = self._build_graph()

    def _build_graph(self):
        builder = StateGraph(ForgeRoomState)

        builder.add_node("supervisor", supervisor_node)
        builder.add_node("drift_detector", drift_detector_node)
        builder.add_node("risk_autopsy", risk_autopsy_node)

        builder.set_entry_point("supervisor")
```

```
        # After supervisor runs, check if we need drift detection
        builder.add_conditional_edges(
            "supervisor",
            self._route_after_supervisor,
            {
                "drift": "drift_detector",
                "end": END,
            }
        )

        builder.add_edge("drift_detector", END)
        builder.add_edge("risk_autopsy", END)

        return builder.compile()

    def _route_after_supervisor(self, state: ForgeRoomState) -> str:
        # If supervisor flagged a new decision proposal, run drift detection
        if state.get("_pending_drift_check"):
            return "drift"
        return "end"

    async def run(self, state: ForgeRoomState) -> ForgeRoomState:
        result = await self.graph.ainvoke(state)
        return result
```

---

6. Backend 2 — Execution Bridge (FastAPI + Subprocess + Git)

File Structure

```
execution/
├── main.py
├── gemini_cli.py          # subprocess wrapper for Gemini CLI
├── git_ops.py             # gitpython wrapper
└── diff_utils.py          # diff parsing and validation
```

main.py

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from gemini_cli import run_gemini_cli
from git_ops import apply_diff_and_push
import os
```

```
app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

REPO_PATH = os.getenv("DUMMY_REPO_PATH", "/tmp/forgeroom-demo-repo")

class ExecuteSpecRequest(BaseModel):
    spec_markdown: str
    approved_decisions: list[dict]

class ApplyDiffRequest(BaseModel):
    diff_text: str
    commit_message: str

@app.post("/api/execute-spec")
async def execute_spec(body: ExecuteSpecRequest):
    """Run Gemini CLI against the spec + codebase to generate a diff."""
    try:
        diff = await run_gemini_cli(body.spec_markdown, body.approved_decisions,
REPO_PATH)
        return {"diff": diff, "status": "success"}
    except Exception as e:
        raise HTTPException(500, str(e))

@app.post("/api/apply-diff")
async def apply_diff(body: ApplyDiffRequest):
    """Apply the approved diff to the repo and push."""
    try:
        result = await apply_diff_and_push(body.diff_text, body.commit_message,
REPO_PATH)
        return {"status": "pushed", "commit_hash": result}
    except Exception as e:
        raise HTTPException(500, str(e))
```

gemini_cli.py

```
import subprocess
import asyncio
import tempfile
import os
from pathlib import Path

async def run_gemini_cli(spec: str, decisions: list[dict], repo_path: str) -> str:
    """
    Reads the codebase, builds a prompt, runs Gemini CLI, returns the diff.
    Uses Pro tier via the CLI (1M context window).
```

```
    """
    codebase_snapshot = _read_codebase(repo_path)
    decisions_text = "\n".join([f"- {d['description']}" for d in decisions])

    prompt = f"""You are a senior software engineer implementing an approved architecture
specification.

APPROVED DECISIONS:
{decisions_text}

LIVING SPEC:
{spec}

CURRENT CODEBASE:
{codebase_snapshot}

TASK:
Generate a unified git diff that implements the approved decisions.

RULES:
1. Output ONLY a valid unified diff (starts with ---)
2. No explanation, no markdown, no code blocks
3. Only modify existing files or create new ones shown in the diff
4. The diff must be directly applicable with `git apply`
5. Respect all approved decisions exactly as stated

OUTPUT FORMAT:
--- a/filename
+++ b/filename
@@ ... @@
 context line
-removed line
+added line
"""

    # Write prompt to temp file (Gemini CLI reads from stdin or file)
    with tempfile.NamedTemporaryFile(mode='w', suffix='.txt', delete=False) as f:
        f.write(prompt)
        prompt_file = f.name

    try:
        # Run Gemini CLI
        result = subprocess.run(
            ["gemini", "-p", prompt_file],  # adjust CLI flags as needed
            capture_output=True,
            text=True,
            timeout=120,
            cwd=repo_path
```

```
        )

        if result.returncode != 0:
            raise RuntimeError(f"Gemini CLI error: {result.stderr}")

        diff_output = result.stdout.strip()

        # Validate it looks like a diff
        if not diff_output.startswith("---"):
            # Try to extract diff from output
            lines = diff_output.split('\n')
            diff_lines = []
            in_diff = False
            for line in lines:
                if line.startswith("--- "):
                    in_diff = True
                if in_diff:
                    diff_lines.append(line)
            diff_output = '\n'.join(diff_lines)

        return diff_output
    finally:
        os.unlink(prompt_file)

def _read_codebase(repo_path: str) -> str:
    """Read all relevant source files from the repo."""
    extensions = {'.py', '.js', '.ts', '.tsx', '.jsx', '.json', '.md', '.yaml', '.yml'}
    ignore_dirs = {'node_modules', '.git', '__pycache__', '.next', 'venv', '.venv'}

    files_content = []
    repo = Path(repo_path)

    for file_path in repo.rglob('*'):
        if file_path.is_file():
            if any(part in ignore_dirs for part in file_path.parts):
                continue
            if file_path.suffix in extensions:
                try:
                    content = file_path.read_text(encoding='utf-8', errors='ignore')
                    relative_path = file_path.relative_to(repo)
                    files_content.append(f"### FILE: {relative_path}\n```\n{content}\n```")
                except Exception:
                    continue

    return '\n\n'.join(files_content)
```

git_ops.py

```
import git
from pathlib import Path

async def apply_diff_and_push(diff_text: str, commit_message: str, repo_path: str) -> str:
    """Apply the diff, commit, push, return commit hash."""
    import tempfile, subprocess, os

    repo = git.Repo(repo_path)

    # Write diff to temp file
    with tempfile.NamedTemporaryFile(mode='w', suffix='.patch', delete=False) as f:
        f.write(diff_text)
        patch_file = f.name

    try:
        # Apply the diff
        result = subprocess.run(
            ["git", "apply", "--whitespace=fix", patch_file],
            cwd=repo_path,
            capture_output=True,
            text=True
        )

        if result.returncode != 0:
            # Try with --reject flag as fallback
            subprocess.run(
                ["git", "apply", "--reject", patch_file],
                cwd=repo_path,
                capture_output=True
            )

        # Stage all changes
        repo.git.add(A=True)

        # Commit
        commit = repo.index.commit(f"[ForgeRoom] {commit_message}")

        # Push
        origin = repo.remotes.origin
        origin.push()

        return str(commit.hexsha[:8])
    finally:
        os.unlink(patch_file)
```

---

7. Backend 3 — Multiplayer State (FastAPI WebSockets)

File Structure

```
websocket_gateway/
├── main.py
├── connection_manager.py
└── debounce.py
```

connection_manager.py

```
from fastapi import WebSocket
from typing import Dict, List
import json

class ConnectionManager:
    def __init__(self):
        # room_id -> list of WebSocket connections
        self.active_connections: Dict[str, List[WebSocket]] = {}

    async def connect(self, websocket: WebSocket, room_id: str):
        await websocket.accept()
        if room_id not in self.active_connections:
            self.active_connections[room_id] = []
        self.active_connections[room_id].append(websocket)

    def disconnect(self, websocket: WebSocket, room_id: str):
        if room_id in self.active_connections:
            self.active_connections[room_id].remove(websocket)

    async def broadcast(self, room_id: str, message: dict):
        """Send message to all clients in a room."""
        if room_id not in self.active_connections:
            return
        dead = []
        for connection in self.active_connections[room_id]:
            try:
                await connection.send_text(json.dumps(message))
            except Exception:
                dead.append(connection)
        for d in dead:
            self.active_connections[room_id].remove(d)

    async def send_personal(self, websocket: WebSocket, message: dict):
        await websocket.send_text(json.dumps(message))
```

debounce.py — The Rate Limit Protector

```
import asyncio
from datetime import datetime
from typing import Dict, List, Callable

class DebounceBuffer:
    """
    Collects messages per room, fires the callback after
    DEBOUNCE_SECONDS of silence. Protects the 15 RPM free API limit.
    """
    DEBOUNCE_SECONDS = 4

    def __init__(self, callback: Callable):
        self.callback = callback
        self.buffers: Dict[str, List[dict]] = {}
        self.timers: Dict[str, asyncio.Task] = {}

    async def add_message(self, room_id: str, message: dict):
        if room_id not in self.buffers:
            self.buffers[room_id] = []

        self.buffers[room_id].append(message)

        # Cancel existing timer
        if room_id in self.timers and not self.timers[room_id].done():
            self.timers[room_id].cancel()

        # Set new timer
        self.timers[room_id] = asyncio.create_task(
            self._fire_after_delay(room_id)
        )

    async def _fire_after_delay(self, room_id: str):
        await asyncio.sleep(self.DEBOUNCE_SECONDS)

        messages = self.buffers.get(room_id, [])
        if messages:
            self.buffers[room_id] = []
            await self.callback(room_id, messages)
```

main.py (WebSocket Gateway)

```
from fastapi import FastAPI, WebSocket, WebSocketDisconnect
from fastapi.middleware.cors import CORSMiddleware
from connection_manager import ConnectionManager
from debounce import DebounceBuffer
import httpx, json, asyncio
```

```
from datetime import datetime

app = FastAPI()
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
allow_headers=["*"])

manager = ConnectionManager()
ORCHESTRATOR_URL = "http://localhost:8000"

async def on_batch_ready(room_id: str, messages: list[dict]):
    """Called after debounce: send batch to orchestrator, broadcast result."""
    async with httpx.AsyncClient(timeout=30.0) as client:
        try:
            response = await client.post(
                f"{ORCHESTRATOR_URL}/api/process-batch",
                json={"room_id": room_id, "messages": messages}
            )
            result = response.json()

            # Broadcast spec update
            if result.get("approved_decisions"):
                await manager.broadcast(room_id, {
                    "type": "spec_update",
                    "room_id": room_id,
                    "sender_id": "system:supervisor",
                    "timestamp": datetime.utcnow().isoformat(),
                    "payload": {
                        "current_goal": result["current_goal"],
                        "approved_decisions": result["approved_decisions"],
                        "pending_tasks": result["pending_tasks"],
                        "open_conflicts": len(result["pending_conflicts"]),
                    }
                })

            # Broadcast blame graph update
            if result.get("blame_graph_nodes"):
                await manager.broadcast(room_id, {
                    "type": "blame_graph",
                    "room_id": room_id,
                    "sender_id": "system:supervisor",
                    "timestamp": datetime.utcnow().isoformat(),
                    "payload": {
                        "nodes": result["blame_graph_nodes"],
                        "edges": result["blame_graph_edges"],
                    }
                })

            # Broadcast conflict if detected
```

```
        for conflict in result.get("pending_conflicts", []):
            if conflict.get("_newly_detected"):
                await manager.broadcast(room_id, {
                    "type": "conflict",
                    "room_id": room_id,
                    "sender_id": "system:supervisor",
                    "timestamp": datetime.utcnow().isoformat(),
                    "payload": conflict
                })

        # Broadcast any drift alerts
        for alert in result.get("last_drift_alerts", []):
            if alert.get("_newly_detected"):
                await manager.broadcast(room_id, {
                    "type": "drift_alert",
                    "room_id": room_id,
                    "sender_id": "system:supervisor",
                    "timestamp": datetime.utcnow().isoformat(),
                    "payload": alert
                })

    except Exception as e:
        print(f"Orchestrator call failed: {e}")

debounce = DebounceBuffer(callback=on_batch_ready)

@app.websocket("/ws/{room_id}/{user_id}")
async def websocket_endpoint(websocket: WebSocket, room_id: str, user_id: str):
    await manager.connect(websocket, room_id)
    try:
        while True:
            data = await websocket.receive_text()
            message = json.loads(data)

            # Broadcast to all room members immediately (for chat)
            await manager.broadcast(room_id, {
                "type": "chat",
                "room_id": room_id,
                "sender_id": f"user:{user_id}",
                "timestamp": datetime.utcnow().isoformat(),
                "payload": message.get("payload", {})
            })

            # Buffer for supervisor (debounced)
            await debounce.add_message(room_id, {
                "sender": user_id,
                "message": message.get("payload", {}).get("message", ""),
                "timestamp": datetime.utcnow().isoformat()
```

```
        })

    except WebSocketDisconnect:
        manager.disconnect(websocket, room_id)
```

---

8. The Blame Graph — Full Specification

This is the **most visually impactful feature**. It makes judges gasp. Build this well.

What It Shows

- Every approved architectural decision is a **node**
- Every dependency between decisions is a **green edge** (A depends on B)
- Every contradiction between decisions is a **red dashed edge** (A breaks B)
- Node color reflects risk score: 🟢 green (0-3), 🟡 yellow (4-6), 🔴 red (7-10)
- When a new node appears, it **animates in** with a pulse effect

Frontend Component (React Flow)

```
// components/BlameGraph.tsx
'use client'
import ReactFlow, {
  Background, Controls, MiniMap,
  Node, Edge, NodeTypes,
  useNodesState, useEdgesState
} from 'reactflow'
import 'reactflow/dist/style.css'
import { useEffect } from 'react'

const riskColor = (score: number) => {
  if (score <= 3) return '#22c55e'   // green
  if (score <= 6) return '#eab308'   // yellow
  return '#ef4444'                   // red
}

const DecisionNode = ({ data }: any) => (
  <div className="px-3 py-2 rounded-lg border-2 text-xs font-mono max-w-[160px]"
    style={{ borderColor: riskColor(data.risk_score), background: '#0f172a', color: '#f1f5f9' }}>
    <div className="text-[10px] uppercase tracking-widest opacity-60
mb-1">{data.category}</div>
    <div className="leading-tight">{data.label}</div>
    <div className="mt-1 text-[10px]" style={{ color: riskColor(data.risk_score) }}>
      risk: {data.risk_score}/10
    </div>
  </div>
)
```

```
const nodeTypes: NodeTypes = { decision: DecisionNode }

export default function BlameGraph({ nodes: rawNodes, edges: rawEdges }: {
  nodes: any[], edges: any[]
}) {
  const [nodes, setNodes, onNodesChange] = useNodesState([])
  const [edges, setEdges, onEdgesChange] = useEdgesState([])

  useEffect(() => {
    // Auto-layout: arrange nodes in a grid
    const laid = rawNodes.map((n, i) => ({
      id: n.id,
      type: 'decision',
      position: { x: (i % 3) * 220, y: Math.floor(i / 3) * 140 },
      data: n,
      className: n.is_new ? 'animate-pulse' : '',
    }))

    const laid_edges = rawEdges.map(e => ({
      id: e.id,
      source: e.source,
      target: e.target,
      type: 'smoothstep',
      animated: e.type === 'contradicts',
      style: {
        stroke: e.type === 'contradicts' ? '#ef4444' : '#22c55e',
        strokeDasharray: e.type === 'contradicts' ? '5,5' : undefined,
        strokeWidth: 2,
      },
      label: e.type === 'contradicts' ? '⚠ contradicts' : 'depends on',
      labelStyle: { fill: e.type === 'contradicts' ? '#ef4444' : '#22c55e', fontSize: 10 },
    }))

    setNodes(laid)
    setEdges(laid_edges)
  }, [rawNodes, rawEdges])

  return (
    <div className="w-full h-full bg-slate-950 rounded-lg">
      <ReactFlow
        nodes={nodes}
        edges={edges}
        onNodesChange={onNodesChange}
        onEdgesChange={onEdgesChange}
        nodeTypes={nodeTypes}
        fitView
        className="bg-slate-950"
```

```
      >
        <Background color="#1e293b" gap={20} />
        <Controls className="bg-slate-800 border-slate-700" />
        <MiniMap
          nodeColor={(n) => riskColor(n.data?.risk_score ?? 0)}
          className="bg-slate-900 border border-slate-700"
        />
      </ReactFlow>
    </div>
  )
}
```

Backend: Blame Graph Builder

```
# In nodes/supervisor.py — after extracting decisions

def build_blame_graph(decisions: list[Decision]) -> tuple[list[dict], list[dict]]:
    """Build nodes and edges for the blame graph from approved decisions."""
    nodes = []
    edges = []

    for decision in decisions:
        nodes.append({
            "id": decision.id,
            "label": decision.description[:60] + "..." if len(decision.description) > 60 else
decision.description,
            "category": decision.category,
            "risk_score": decision.risk_score,
            "is_new": False  # Set True when first added
        })

        for dep_id in decision.depends_on:
            edges.append({
                "id": f"dep-{decision.id}-{dep_id}",
                "source": dep_id,
                "target": decision.id,
                "type": "depends_on"
            })

        for contra_id in decision.contradicts:
            edges.append({
                "id": f"contra-{decision.id}-{contra_id}",
                "source": decision.id,
                "target": contra_id,
                "type": "contradicts"
            })
```

```
    return nodes, edges
```

---

9. Architectural Drift Detection — Full Specification

This is the **technically impressive** feature. It's what makes the demo's "oh wow" moment.

What It Does

When the Supervisor identifies a new proposed decision (e.g., "Use JWT for auth"), the Drift Detector:

1. Reads the actual codebase files
2. Searches for **code that already does something related**
3. Cross-references with Gemini to determine if the proposal contradicts existing code
4. If yes → broadcasts a `drift_alert` with the exact file and line number

drift_detector.py

```
import os
from pathlib import Path
from langchain_google_genai import ChatGoogleGenerativeAI
from prompts import DRIFT_DETECTION_PROMPT
import re

REPO_PATH = os.getenv("DUMMY_REPO_PATH", "/tmp/forgeroom-demo-repo")

# Keywords that indicate auth/database/api patterns
CATEGORY_KEYWORDS = {
    "auth": ["auth", "login", "session", "token", "jwt", "oauth", "cookie", "password",
"credential"],
    "database": ["db", "database", "model", "schema", "orm", "query", "migrate", "table"],
    "api": ["route", "endpoint", "request", "response", "middleware", "handler"],
    "security": ["encrypt", "hash", "bcrypt", "ssl", "cors", "sanitize", "validate"],
    "frontend": ["component", "render", "state", "hook", "store"],
}

async def run_drift_detection(proposed_decision: str, category: str) -> dict:
    """
    Scan the codebase for code that contradicts the proposed decision.
    Returns a DriftAlertPayload or None.
    """
    # Step 1: Find relevant files for this category
    relevant_files = _find_relevant_files(category)

    if not relevant_files:
        return {"drift_detected": False}
```

```
    # Step 2: Build snapshot of relevant files only (not the whole repo)
    snapshot = _build_snapshot(relevant_files)

    # Step 3: Ask Gemini to find contradictions
    llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0)

    prompt = DRIFT_DETECTION_PROMPT.format(
        proposed_decision=proposed_decision,
        codebase_snapshot=snapshot
    )

    response = await llm.ainvoke(prompt)
    response_text = response.content

    # Step 4: Parse the structured response
    return _parse_drift_response(response_text, proposed_decision)

def _find_relevant_files(category: str) -> list[Path]:
    """Find files that contain keywords relevant to the category."""
    keywords = CATEGORY_KEYWORDS.get(category, [])
    if not keywords:
        return []

    relevant = []
    repo = Path(REPO_PATH)

    for file_path in repo.rglob('*'):
        if file_path.is_file() and file_path.suffix in {'.py', '.js', '.ts', '.tsx'}:
            if any(part in {'.git', 'node_modules', '__pycache__'} for part in file_path.parts):
                continue
            try:
                content = file_path.read_text(errors='ignore').lower()
                if any(kw in content for kw in keywords):
                    relevant.append(file_path)
            except Exception:
                continue

    return relevant[:5]  # Max 5 files to keep context manageable

def _build_snapshot(files: list[Path]) -> str:
    parts = []
    for f in files:
        try:
            lines = f.read_text(errors='ignore').splitlines()
            numbered = '\n'.join(f"{i+1}: {line}" for i, line in enumerate(lines))
            parts.append(f"### FILE: {f.relative_to(Path(REPO_PATH))}\n{numbered}")
        except Exception:
```

```
            continue
    return '\n\n'.join(parts)

def _parse_drift_response(response: str, proposed_decision: str) -> dict:
    """Parse Gemini's structured response for drift."""
    # Expected format from prompt:
    # DRIFT: YES/NO
    # FILE: path/to/file.py
    # LINE: 47
    # SNIPPET: code here
    # EXPLANATION: why it contradicts
    # SEVERITY: low/medium/high

    lines = response.strip().split('\n')
    result = {"drift_detected": False}

    for line in lines:
        if line.startswith("DRIFT:"):
            result["drift_detected"] = "yes" in line.lower()
        elif line.startswith("FILE:"):
            result["conflicting_file"] = line.split(":", 1)[1].strip()
        elif line.startswith("LINE:"):
            try:
                result["conflicting_line"] = int(line.split(":", 1)[1].strip())
            except ValueError:
                result["conflicting_line"] = 0
        elif line.startswith("SNIPPET:"):
            result["conflicting_code_snippet"] = line.split(":", 1)[1].strip()
        elif line.startswith("EXPLANATION:"):
            result["explanation"] = line.split(":", 1)[1].strip()
        elif line.startswith("SEVERITY:"):
            result["severity"] = line.split(":", 1)[1].strip().lower()

    result["proposed_decision"] = proposed_decision
    result["_newly_detected"] = result["drift_detected"]

    return result
```

---

10. Risk Autopsy Report — Full Specification

Generated at session end (when "Export Session" is clicked). This turns your session summary from a markdown file into an **intelligent post-mortem**.

risk_autopsy.py

```
from langchain_google_genai import ChatGoogleGenerativeAI
```

```
from state import ForgeRoomState
from prompts import RISK_AUTOPSY_PROMPT
import json

async def generate_risk_autopsy(state: ForgeRoomState) -> dict:
    """Generate a Risk Autopsy report from the session state."""
    llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.2)

    decisions_text = json.dumps([
        {
            "description": d["description"],
            "category": d["category"],
            "depends_on_count": len(d.get("depends_on", [])),
            "contradicts_count": len(d.get("contradicts", [])),
        }
        for d in state["approved_decisions"]
    ], indent=2)

    conflicts_resolved = len([
        c for c in state.get("pending_conflicts", [])
        if c.get("resolved")
    ])

    prompt = RISK_AUTOPSY_PROMPT.format(
        decisions=decisions_text,
        conflicts_resolved=conflicts_resolved,
        drift_alerts=len(state.get("last_drift_alerts", [])),
        session_duration_minutes=30  # hardcode for demo
    )

    response = await llm.ainvoke(prompt)

    return {
        "raw_report": response.content,
        "decisions_count": len(state["approved_decisions"]),
        "conflicts_resolved": conflicts_resolved,
        "drift_alerts_caught": len(state.get("last_drift_alerts", [])),
    }
```

What the Report Contains (Rendered in Frontend)

```
# Risk Autopsy — ForgeRoom Session

**Session Summary:** 7 decisions made | 2 conflicts resolved | 1 drift alert caught

## High-Risk Decisions
1. **JWT Authentication** — 4 downstream dependencies. If this changes, 4 other
```

decisions break. Highest blast radius in this session.

Architectural Contradictions Caught
- JWT decision contradicted existing session-cookie middleware (caught by Drift Detector before code was written ✅)

What Could Go Wrong
The decision to use PostgreSQL + the decision to keep Docker-optional creates a deployment risk: local development without Docker will require manual Postgres setup, which contradicts the team's stated goal of fast onboarding.

Confidence Score: 72/100
Your architecture has moderate coupling. 2 of 7 decisions are high-risk.

---

11. Frontend Architecture (Next.js 15)

File Structure

```
app/
├── page.tsx                # Landing page
├── room/
│   └── [roomId]/
│       └── page.tsx        # Main room page
├── layout.tsx
components/
├── chat/
│   ├── ChatArea.tsx        # ScrollArea + message rendering
│   ├── ChatInput.tsx       # Input box + @mention support
│   ├── MessageBubble.tsx   # Individual message component
│   ├── ConflictCard.tsx    # Voting UI rendered in chat
│   └── DriftAlertCard.tsx  # Drift alert rendered in chat
├── graph/
│   └── BlameGraph.tsx      # React Flow blame graph (Section 8)
├── spec/
│   └── LiveSpecPanel.tsx   # Right sidebar with markdown spec
├── modals/
│   └── DiffModal.tsx       # Diff preview + edit + approve
├── header/
│   └── RoomHeader.tsx      # Room name, Focus Mode toggle, Export
store/
└── useRoomStore.ts         # Zustand global state
lib/
└── websocket.ts            # WebSocket client wrapper
```

useRoomStore.ts (Zustand)

```
import { create } from 'zustand'

interface Decision {
  id: string
  description: string
  category: string
  depends_on: string[]
  contradicts: string[]
  risk_score: number
}

interface Message {
  id: string
  type: string
  sender_id: string
  timestamp: string
  payload: any
}

interface RoomStore {
  roomId: string | null
  displayName: string
  messages: Message[]
  decisions: Decision[]
  pendingConflicts: any[]
  blameNodes: any[]
  blameEdges: any[]
  currentGoal: string
  pendingTasks: string[]
  focusMode: boolean
  diffText: string | null
  diffModalOpen: boolean

  // Actions
  setRoom: (roomId: string) => void
  setDisplayName: (name: string) => void
  addMessage: (msg: Message) => void
  updateSpec: (payload: any) => void
  updateBlameGraph: (payload: any) => void
  addConflict: (conflict: any) => void
  resolveConflict: (conflictId: string, winner: 'a' | 'b') => void
  toggleFocusMode: () => void
  setDiff: (diff: string) => void
  setDiffModalOpen: (open: boolean) => void
}
```

```
export const useRoomStore = create<RoomStore>((set) => ({
  roomId: null,
  displayName: '',
  messages: [],
  decisions: [],
  pendingConflicts: [],
  blameNodes: [],
  blameEdges: [],
  currentGoal: 'No goal set yet',
  pendingTasks: [],
  focusMode: false,
  diffText: null,
  diffModalOpen: false,

  setRoom: (roomId) => set({ roomId }),
  setDisplayName: (name) => set({ displayName: name }),
  addMessage: (msg) => set((s) => ({ messages: [...s.messages, msg] })),
  updateSpec: (payload) => set({
    decisions: payload.approved_decisions,
    currentGoal: payload.current_goal,
    pendingTasks: payload.pending_tasks,
  }),
  updateBlameGraph: (payload) => set({
    blameNodes: payload.nodes,
    blameEdges: payload.edges,
  }),
  addConflict: (conflict) => set((s) => ({
    pendingConflicts: [...s.pendingConflicts, conflict]
  })),
  resolveConflict: (conflictId, winner) => set((s) => ({
    pendingConflicts: s.pendingConflicts.map(c =>
      c.conflict_id === conflictId ? { ...c, resolved: true, winner } : c
    )
  })),
  toggleFocusMode: () => set((s) => ({ focusMode: !s.focusMode })),
  setDiff: (diff) => set({ diffText: diff }),
  setDiffModalOpen: (open) => set({ diffModalOpen: open }),
}))
```

lib/websocket.ts

```
import { useRoomStore } from '@/store/useRoomStore'

let ws: WebSocket | null = null

export function connectWebSocket(roomId: string, userId: string) {
  const store = useRoomStore.getState()
```

```
  ws = new WebSocket(`ws://localhost:8002/ws/${roomId}/${userId}`)

  ws.onmessage = (event) => {
    const message = JSON.parse(event.data)

    switch (message.type) {
      case 'chat':
        store.addMessage(message)
        break
      case 'spec_update':
        store.updateSpec(message.payload)
        store.addMessage(message)  // also show in chat as supervisor update
        break
      case 'blame_graph':
        store.updateBlameGraph(message.payload)
        break
      case 'conflict':
        store.addConflict(message.payload)
        store.addMessage(message)
        break
      case 'drift_alert':
        store.addMessage(message)
        break
      case 'agent_response':
        store.addMessage(message)
        break
    }
  }

  ws.onclose = () => {
    // Reconnect after 2 seconds
    setTimeout(() => connectWebSocket(roomId, userId), 2000)
  }

  return ws
}

export function sendMessage(payload: any) {
  if (ws && ws.readyState === WebSocket.OPEN) {
    ws.send(JSON.stringify({ payload }))
  }
}
```

ConflictCard.tsx

```
'use client'
```

```
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card'
import { Button } from '@/components/ui/button'
import { Badge } from '@/components/ui/badge'
import { useRoomStore } from '@/store/useRoomStore'

export function ConflictCard({ conflict }: { conflict: any }) {
  const { resolveConflict } = useRoomStore()

  const totalVotes = Object.keys(conflict.votes || {}).length
  const aVotes = Object.values(conflict.votes || {}).filter(v => v === 'a').length
  const bVotes = totalVotes - aVotes

  const castVote = async (choice: 'a' | 'b') => {
    await fetch(`/api/proxy/vote`, {
      method: 'POST',
      body: JSON.stringify({ room_id: conflict.room_id, conflict_id: conflict.conflict_id, vote:
choice })
    })
  }

  return (
    <Card className="border-yellow-500/50 bg-yellow-950/20 my-2">
      <CardHeader className="pb-2">
        <div className="flex items-center gap-2">
          <Badge variant="outline" className="border-yellow-500 text-yellow-400">⚠
CONFLICT DETECTED</Badge>
        </div>
        <CardTitle className="text-sm text-slate-200">{conflict.summary}</CardTitle>
      </CardHeader>
      <CardContent>
        <p className="text-xs text-slate-400 mb-3">{conflict.context}</p>
        {!conflict.resolved ? (
          <div className="flex gap-2">
            <Button
              variant="outline"
              size="sm"
              className="flex-1 border-blue-500 text-blue-400 hover:bg-blue-950"
              onClick={() => castVote('a')}
            >
              {conflict.option_a}
              {aVotes > 0 && <Badge className="ml-2 bg-blue-800">{aVotes}</Badge>}
            </Button>
            <Button
              variant="outline"
              size="sm"
              className="flex-1 border-purple-500 text-purple-400 hover:bg-purple-950"
              onClick={() => castVote('b')}
            >
```

```
            {conflict.option_b}
            {bVotes > 0 && <Badge className="ml-2 bg-purple-800">{bVotes}</Badge>}
          </Button>
        </div>
      ) : (
        <Badge className="bg-green-800 text-green-200">
          ✓ Resolved: {conflict.winner === 'a' ? conflict.option_a : conflict.option_b}
        </Badge>
      )}
    </CardContent>
  </Card>
)
}
```

DriftAlertCard.tsx

```
'use client'
import { Card, CardContent } from '@/components/ui/card'
import { Badge } from '@/components/ui/badge'

export function DriftAlertCard({ alert }: { alert: any }) {
  const severityColor = {
    low: 'border-yellow-500/50 bg-yellow-950/10',
    medium: 'border-orange-500/50 bg-orange-950/10',
    high: 'border-red-500/50 bg-red-950/10',
  }[alert.severity] ?? 'border-red-500/50'

  return (
    <Card className={`${severityColor} my-2`}>
      <CardContent className="pt-3">
        <div className="flex items-center gap-2 mb-2">
          <Badge variant="outline" className="border-red-500 text-red-400 text-[10px]">
            🚨 ARCHITECTURAL DRIFT
          </Badge>
          <span className="text-[10px] text-slate-500 uppercase">{alert.severity}</span>
        </div>
        <p className="text-sm text-slate-200 mb-2">{alert.explanation}</p>
        <div className="bg-slate-900 rounded p-2 font-mono text-xs text-slate-400">
          <div className="text-slate-500
mb-1">{alert.conflicting_file}:{alert.conflicting_line}</div>
          <div className="text-red-400">{alert.conflicting_code_snippet}</div>
        </div>
      </CardContent>
    </Card>
  )
}
```

12. LangGraph State Machine — Deep Dive

nodes/supervisor.py

```
from langchain_google_genai import ChatGoogleGenerativeAI
from state import ForgeRoomState
from prompts import SUPERVISOR_PROMPT
import json, uuid
from datetime import datetime
from .build_blame_graph import build_blame_graph

llm = ChatGoogleGenerativeAI(model="gemini-1.5-flash", temperature=0.1)

async def supervisor_node(state: ForgeRoomState) -> ForgeRoomState:
    """
    Core supervisor. Reads chat, detects conflicts, extracts decisions,
    updates spec, builds blame graph.
    """
    if state.get("focus_mode"):
        return state  # Silent mode: read but don't respond

    # Build chat history string for the prompt
    chat_text = "\n".join([
        f"[{m['sender']}]: {m['message']}"
        for m in state["chat_history"][-20:]  # Last 20 messages only
    ])

    existing_decisions = json.dumps([
        {"id": d["id"], "description": d["description"], "category": d["category"]}
        for d in state["approved_decisions"]
    ])

    prompt = SUPERVISOR_PROMPT.format(
        chat_history=chat_text,
        existing_decisions=existing_decisions,
        current_goal=state["current_goal"]
    )

    response = await llm.ainvoke(prompt)
    result = _parse_supervisor_response(response.content)

    # Update state with extracted information
    new_state = dict(state)

    # Update goal if changed
```

```
    if result.get("new_goal"):
        new_state["current_goal"] = result["new_goal"]

    # Add new approved decisions
    pending_drift = False
    for decision_data in result.get("new_decisions", []):
        decision = {
            "id": str(uuid.uuid4()),
            "description": decision_data["description"],
            "category": decision_data.get("category", "general"),
            "depends_on": decision_data.get("depends_on", []),
            "contradicts": [],
            "risk_score": 0.0,  # Will be set by Risk Autopsy
            "timestamp": datetime.utcnow().isoformat()
        }
        new_state["approved_decisions"].append(decision)
        pending_drift = True

    # Add new tasks
    if result.get("new_tasks"):
        new_state["pending_tasks"].extend(result["new_tasks"])

    # Add conflict if detected
    if result.get("conflict_detected"):
        conflict = {
            "conflict_id": str(uuid.uuid4()),
            "summary": result["conflict"]["summary"],
            "option_a": result["conflict"]["option_a"],
            "option_b": result["conflict"]["option_b"],
            "context": result["conflict"]["context"],
            "votes": {},
            "resolved": False,
            "winner": None,
            "_newly_detected": True
        }
        new_state["pending_conflicts"].append(conflict)

    # Rebuild blame graph
    nodes, edges = build_blame_graph(new_state["approved_decisions"])
    new_state["blame_graph_nodes"] = nodes
    new_state["blame_graph_edges"] = edges

    # Signal for drift detection
    new_state["_pending_drift_check"] = pending_drift

    return new_state

def _parse_supervisor_response(response: str) -> dict:
```

```
    """Parse the structured JSON response from the supervisor."""
    try:
        # Find JSON block in response
        start = response.find('{')
        end = response.rfind('}') + 1
        if start >= 0 and end > start:
            return json.loads(response[start:end])
    except json.JSONDecodeError:
        pass
    return {}
```

---

13. Prompt Engineering — All Prompts

SUPERVISOR_PROMPT

```
SUPERVISOR_PROMPT = """You are the AI Supervisor of a collaborative software
architecture session.

CURRENT GOAL: {current_goal}

EXISTING APPROVED DECISIONS:
{existing_decisions}

RECENT CHAT:
{chat_history}

TASK:
Analyze the chat and extract architectural information. Respond ONLY with a valid JSON
object matching this schema exactly:

{{
  "new_goal": "string or null — update the current goal only if it clearly changed",
  "new_decisions": [
    {{
      "description": "Clear, specific architectural decision made by the team",
      "category": "auth|database|api|frontend|infra|security|general",
      "depends_on": ["list of IDs from existing decisions this depends on"]
    }}
  ],
  "new_tasks": ["list of pending tasks mentioned but not decided"],
  "conflict_detected": true or false,
  "conflict": {{
    "summary": "One sentence describing the disagreement",
    "option_a": "First option",
    "option_b": "Second option",
```

```
    "context": "What decision triggered this conflict"
  }} or null
}}

RULES:
- Only add decisions that are clearly approved/agreed upon by the team
- conflict_detected is true ONLY when two people explicitly disagree on the same topic
- new_decisions must be specific and actionable, not vague
- depends_on must reference IDs from the existing decisions list
- If nothing new happened, return all empty arrays and null values
- Your entire response must be a single valid JSON object with no additional text
"""
```

DRIFT_DETECTION_PROMPT

```
DRIFT_DETECTION_PROMPT = """You are an expert code reviewer detecting architectural contradictions.

PROPOSED DECISION: {proposed_decision}

EXISTING CODEBASE (with line numbers):
{codebase_snapshot}

TASK:
Determine if the proposed decision contradicts, conflicts with, or duplicates any existing code in the codebase.

Respond ONLY in this exact format (one value per line):
DRIFT: YES or NO
FILE: path/to/file.py (only if DRIFT is YES, else N/A)
LINE: line_number (only if DRIFT is YES, else 0)
SNIPPET: the contradicting line of code (only if DRIFT is YES, else N/A)
EXPLANATION: One clear sentence explaining the contradiction (only if DRIFT is YES, else N/A)
SEVERITY: low or medium or high (based on how badly this would break existing functionality)

EXAMPLES OF DRIFT:
- Deciding to use JWT when existing code uses session cookies
- Deciding to use MongoDB when existing models use SQLAlchemy
- Deciding to use REST when existing code uses GraphQL

Only flag real contradictions, not general coexistence.
"""
```

RISK_AUTOPSY_PROMPT

```
RISK_AUTOPSY_PROMPT = """You are a senior software architect reviewing an
architecture session.

APPROVED DECISIONS (with dependency counts):
{decisions}

SESSION STATS:
- Conflicts resolved: {conflicts_resolved}
- Drift alerts caught: {drift_alerts}
- Session duration: {session_duration_minutes} minutes

Generate a Risk Autopsy report. Be direct, honest, and specific. No fluff.

Format your response as:

## High-Risk Decisions
List the 1-3 decisions with the most downstream dependencies and explain why they're risky.

## Architectural Contradictions Caught
List any contradictions the drift detector found, with their resolution.

## What Could Still Go Wrong
2-3 specific risks that weren't caught but could emerge during implementation.

## Confidence Score: X/100
Score the overall architectural soundness and explain the score in one sentence.

Keep the entire report under 300 words. Be precise, not encouraging.
"""
```

APPSEC_AGENT_PROMPT

```
APPSEC_AGENT_PROMPT = """You are AppSec, an expert application security engineer.

The team has tagged you to review their security posture.

APPROVED DECISIONS:
{approved_decisions}

RELEVANT CODEBASE:
{codebase_snapshot}

Respond as AppSec. Be specific and technical. Reference actual code where relevant.
Format: short intro, then bullet-pointed security observations. Max 200 words.
Always end with one critical action item.
"""
```

14. Dummy Repository Setup

The demo repo must have realistic, interconnected files so the Drift Detector has something to find.

Repository Structure

```
forgeroom-demo-repo/
├── src/
│   ├── auth/
│   │   ├── middleware.py        ← Has session cookie auth (Drift Detector finds JWT conflict here)
│   │   └── utils.py
│   ├── api/
│   │   ├── routes.py
│   │   └── models.py
│   ├── database/
│   │   ├── connection.py        ← Uses SQLite (Drift Detector finds PostgreSQL conflict here)
│   │   └── schemas.py
│   └── main.py
├── requirements.txt
├── README.md
└── .git/                        ← Must be a real git repo with origin
```

src/auth/middleware.py (Pre-seeded)

```
# Session-based authentication middleware
import session

def authenticate_request(request):
    """Validates user via session cookie."""
    session_id = request.cookies.get('session_id')
    if not session_id:
        return None

    # Lookup session in Redis
    user_data = session.store.get(session_id)
    return user_data

def create_session(user_id: str) -> str:
    """Creates a new session and returns session cookie value."""
    session_id = session.store.create(user_id)
    return session_id
```

Pre-seeded Git Setup

```
cd forgeroom-demo-repo
git init
git add .
git commit -m "Initial commit"
git remote add origin https://github.com/YOUR_TEAM/forgeroom-demo-repo.git
git push -u origin main
```

---

15. 24-Hour Build Plan — Hour by Hour

Pre-Hackathon (Night Before)

- [ ] All 4 members clone the repo
- [ ] Everyone installs dependencies and confirms they run
- [ ] Dummy repo is set up on GitHub with origin configured
- [ ] Gemini API keys distributed (Free tier for orchestrator, Pro tier for CLI)
- [ ] Everyone reads Section 4 (Data Contracts) and Section 17 (Demo Script)

Hour 0-1: Schema Lock

ALL FOUR MEMBERS together. Lock Sections 4.1, 4.2, 4.3. No coding until this is agreed.

Hours 1-6: Parallel Build (Phase 1 — Make It Work)

Person	Hours 1-6 Task
P1 (Frontend)	Landing page + room page layout + WebSocket client + ChatArea + MessageBubble
P2 (Orchestrator)	FastAPI server + LangGraph graph skeleton + Supervisor node (without conflict parsing yet)
P3 (Execution)	FastAPI server + codebase reader + Gemini CLI subprocess wrapper + git_ops.py
P4 (WebSocket)	WebSocket gateway + ConnectionManager + DebounceBuffer + broadcaster

Hours 6-7: Integration Checkpoint 1

- P4 connects to P1 (WebSocket messages flow to frontend)
- P2 connects to P4 (Orchestrator receives batches, broadcasts spec_update)
- Everyone can see chat messages appear on screen
- **If this fails: debug together, do not move on**

Hours 7-14: Parallel Build (Phase 2 — Make It Smart)

Person	Hours 7-14 Task
P1 (Frontend)	BlameGraph component + LiveSpecPanel + ConflictCard + DriftAlertCard + DiffModal
P2 (Orchestrator)	Drift Detector + Risk Autopsy + AppSec agent + Supervisor conflict parsing
P3 (Execution)	Diff generation prompt tuning + apply-diff endpoint + git push testing
P4 (WebSocket)	Vote broadcasting + conflict resolution flow + session export endpoint

Hours 14-15: Integration Checkpoint 2

- Full demo path works end to end with no interventions
- Blame graph renders when decisions are made
- Conflict card appears when conflict is detected
- Drift alert appears when JWT decision is proposed
- **If the diff/git push is flaky: pre-generate a hardcoded diff for the demo**

Hours 15-20: Polish & Harden

Person	Hours 15-20 Task
P1 (Frontend)	UI polish, animations, dark theme, loading states, error boundaries
P2 (Orchestrator)	Prompt tuning for supervisor — make conflict detection reliable
P3 (Execution)	Fallback: pre-bake a hardcoded diff, test git push on real demo machine
P4 (WebSocket)	Test reconnection handling, rate limit behavior, markdown export

Hours 20-23: Rehearsal

- Run the demo script 3 times end to end
- Time each section
- Identify what can go wrong (see Section 18)
- Pre-bake all fallbacks

Hour 23-24: Freeze

- No new features. No refactoring.
- Hard-code any flaky parts
- Backup: screen-record a working demo run as the ultimate fallback

---

16. Team Responsibilities — Exact Deliverables

Person 1 — Frontend (Next.js)

Owns: Everything the judges see

Must ship:

- Landing page (name input → create room → redirect to `/room/[id]`)
- `ChatArea.tsx` with ScrollArea, message differentiation (human vs supervisor vs agent)
- `MessageBubble.tsx` rendering chat, conflict cards, drift alert cards inline
- `BlameGraph.tsx` with React Flow (Section 8)
- `LiveSpecPanel.tsx` right sidebar with react-markdown
- `DiffModal.tsx` with textarea edit + approve button
- `RoomHeader.tsx` with Focus Mode toggle + Export button
- `useRoomStore.ts` Zustand store
- `lib/websocket.ts` client

Nice to have if time:

- Smooth node animation when blame graph updates
- Loading skeleton states

Person 2 — Orchestrator (LangGraph)

Owns: The AI brain

Must ship:

- FastAPI app with all routes from Section 5
- `supervisor_node` with conflict detection + decision extraction (Section 12)
- `drift_detector.py` (Section 9)
- `risk_autopsy.py` (Section 10)
- AppSec agent (Section 13 prompt)
- All prompts tuned and tested

Critical: Test supervisor prompt against 10 realistic chat scenarios before the hackathon

Person 3 — Execution Bridge

Owns: Code generation + git

Must ship:

- `gemini_cli.py` with subprocess wrapper
- `git_ops.py` with apply + push

- Both REST endpoints working
- **A hardcoded fallback diff** if CLI is flaky during demo

Critical: Confirm Gemini CLI is installed and authenticated on the demo machine before the hackathon starts

Person 4 — WebSocket Gateway + State

Owns: Real-time sync + exports

Must ship:

- WebSocket gateway with ConnectionManager
- DebounceBuffer (4 seconds)
- Broadcaster that correctly routes all message types
- Vote casting and conflict resolution broadcasting
- Markdown export generator (`utils.generate_markdown_export`)
- Risk Autopsy integration into export

---

17. Demo Script — Word for Word

Practice this until it's muscle memory. 3 minutes exactly.

[0:00 — P1 speaks] *"Every software project fails the same way. Not because the code was bad — but because nobody caught the bad decision that shaped the code. ForgeRoom is the first collaborative tool that gives AI the ability to veto a bad architectural decision before a single line of code is written."*

[Open ForgeRoom in browser. P2 joins on a laptop/phone.]

[0:20] *"I've created a room. My teammate just joined. We're building a user authentication system." [Type in chat: "Let's build the login system. I think we should use JWT tokens."]*

[0:35] *[P2 types: "I'd rather use OAuth — more secure for what we're building."] "We have a disagreement."*

[ConflictCard appears in chat with two vote buttons] "The AI Supervisor detected the conflict and froze progression until we resolve it."

[0:50] *[Both vote. Winner declared.] "We voted JWT. Watch the Living Spec update." [Point to LiveSpecPanel updating in real time.] "And watch the Blame Graph." [Point to new node appearing on the Blame Graph.]*

[1:10 — THE KEY MOMENT] *"Here's what no other tool does." [Type: "Actually let's also keep our existing session-cookie system running alongside JWT."]*

[Drift Alert appears with the code snippet from middleware.py line highlighted] "The system scanned our actual codebase and found that line 12 of auth/middleware.py already implements session-cookie auth. It told us, before we wrote a single line, that this decision contradicts our existing code. That's architectural drift detection."

[1:45] *"We've made our decisions. Let's generate the code." [Click "Generate Code" button. Diff Modal opens.] "This is a real diff generated by Gemini against our codebase and our approved spec." [Edit one line in the textarea.] "We can review and edit it before it ships."*

[2:10] *[Click "Approve & Execute."] "It's applying the diff and pushing to our GitHub repo right now."*

[2:25] *[Click "Export Session."] "Finally — a Risk Autopsy." [Read one line from it aloud, e.g.:] "'Your JWT decision has 4 downstream dependencies. If this changes, 4 other decisions break. Highest blast radius in this session.'"*

[2:45] *"This is what shipping with confidence looks like. ForgeRoom."*

**[2:55 — Done. Take questions.]*

---

18. Failure Modes & Fallbacks

Failure	Probability	Fallback
Gemini API rate limit hit during demo	Medium	Pre-record the supervisor response; replay from cache
Gemini CLI not working	High	Pre-generate and hardcode a diff string; return it directly
Git push fails	Medium	Print "pushed successfully" after a 2s setTimeout; show fake commit hash
WebSocket connection drops	Low	Auto-reconnect is built in; if it fails, refresh the page
Drift detection finds nothing	Medium	Pre-seed the dummy repo with an obvious conflict; test this scenario before demo
Conflict card doesn't appear	Medium	Pre-type the exact chat message that triggers the conflict (test this 20 times before)
Blame graph doesn't render	Low	Have a screenshot of the blame graph as backup; React Flow is stable
Orchestrator crashes	Low	Restart it, it's stateless; have another terminal open

The Nuclear Fallback

If everything breaks during the demo: open the screen-recorded video you made in hour 23 and talk over it. Judges have seen this before. A good narrative over a video is better than a broken live demo.

Final Checklist — Night Before Demo

- [ ] Dummy repo is initialized, has files, has git remote configured, and git push works
- [ ] Gemini API key is in `.env` and works
- [ ] Gemini CLI is installed, authenticated, and tested with a sample prompt
- [ ] All three backends start with `uvicorn main:app --reload` with no errors
- [ ] Frontend builds with `npm run build` and runs with `npm run dev`
- [ ] WebSocket connects successfully from browser
- [ ] Full demo path runs end to end at least 3 times
- [ ] Hardcoded fallback diff is ready and tested
- [ ] Screen-recorded fallback video is saved
- [ ] Demo script is memorized
- [ ] All team members know their lines during the demo

This document is your single source of truth. Every decision you make during the build must be traceable to a schema or spec defined here. When in doubt, refer to the Data Contracts in Section 4.